

Prompt Engineering Mini-Assessment

Session 7 — Multi-Step Prompt Workflow (Extract → Reason → Format)

Workflow Design

This assessment implements the Extract → Reason → Format design pattern from Session 7, combining three prompting techniques from the module into a single working pipeline for support-ticket triage. Each step uses the technique best suited to its job: few-shot prompting to anchor extraction format, chain-of-thought to reason through an urgency judgment, and JSON mode to produce a clean, parseable final output.

Pipeline: Raw ticket → Step 1 (few-shot extraction) → Step 2 (CoT urgency reasoning) → Step 3 (JSON formatting)

Step 1 — Extract Entities (Few-Shot)

Technique: few-shot prompting. Two worked examples anchor the exact bullet format so the model doesn't drift on a new ticket.

```
Extract the key entities from a support ticket as a bullet list.
Include: product/service mentioned, issue type, customer impact,
and any dates/deadlines mentioned.
```

```
Ticket: "My card was charged twice for the same order #4521 placed
yesterday. I need this refunded before my statement closes on the 10th."
Entities:
- Product/Service: Billing / Order #4521
- Issue Type: Duplicate charge
- Customer Impact: Financial (double payment)
- Deadline: Statement closes on the 10th
```

```
Ticket: "The app crashes every time I try to upload a photo. Been
happening since the last update."
Entities:
- Product/Service: Mobile app - photo upload feature
- Issue Type: Crash / bug
- Customer Impact: Feature unusable
- Deadline: None mentioned
```

```
Ticket: "{new_ticket_text}"
Entities:
```

Step 2 — Reason Urgency Level (Chain of Thought)

Technique: zero-shot CoT. The model reasons explicitly over four urgency factors before committing to a label, conditioned on Step 1's real output.

```
Given the following extracted entities from a support ticket:
{step1_output}
```

```
Think step by step about how urgent this ticket is, considering:
- Is money or security at risk?
- Is the customer fully blocked from using the product?
- Is there a time-sensitive deadline?
- Is this a one-off inconvenience or a recurring/blocking failure?
```

Reason through each factor, then conclude with a final urgency level:
low, medium, or high.

Step 3 — Format as JSON (JSON Mode)

Technique: schema-forced JSON mode. Both prior outputs are fed in as dynamic input; the instruction forbids any surrounding prose so the result parses directly.

Convert the following into a single valid JSON object with exactly these keys: "entities" (array of strings), "urgency" (string: low/medium/high), "summary" (one-sentence string).

Extracted entities:
{step1_output}

Reasoning and urgency conclusion:
{step2_output}

Return ONLY the JSON object, no other text.

End-to-End Run (Worked Example)

Raw ticket: “My card was charged twice for order #4521 placed yesterday. I need this refunded before my statement closes on the 10th.”

Step 1 output (few-shot extraction):

- Product/Service: Billing / Order #4521
- Issue Type: Duplicate charge
- Customer Impact: Financial (double payment)
- Deadline: Statement closes on the 10th

Step 2 output (CoT reasoning):

1. Money is directly at risk - the customer was double-charged.
 2. This isn't a blocking product failure, but it's a financial harm, which weighs heavily.
 3. There's a hard deadline (statement closing on the 10th) - time-sensitive.
 4. This is a one-off event, but combined with the financial + deadline factors, it needs prompt handling.
- Conclusion: urgency = high

Step 3 output (final JSON):

```
{
  "entities": ["Order #4521", "Duplicate charge",
    "Financial impact", "Deadline: 10th"],
  "urgency": "high",
  "summary": "Customer was double-charged for order #4521 and needs
    a refund before their statement closes on the 10th."
}
```

Rubric Self-Check

Requirement	How it's met
-------------	--------------

Correct technique per step	Few-shot (extraction, anchored by 2 examples) → CoT (explicit stepwise reasoning) → JSON mode (schema-forced, no prose).
Step 2 conditions on Step 1's real output	The entities list from Step 1 is passed directly into the Step 2 prompt, not hardcoded or ignored.
Final JSON is valid and parses cleanly	Output contains exactly the 3 required keys (entities, urgency, summary) and parses via <code>json.loads</code> with no manual cleanup.
Workflow tested end-to-end	One full example run above, with all three intermediate outputs shown for grading transparency.